

School of Information and Physical Sciences
COMP2240/COMP6240 - Operating Systems

Assignment 1 (10%)

Submit using Canvas by **11:59 pm, Friday 06th Sept 2024**

1. Introduction

Assignment 1 will focus on your understanding and ability to successfully implement, execute and compare the results of a number of scheduling algorithms.

2. Assignment Task

Write a program that simulates **First Come First Serve (FCFS)**, **Shortest Remaining Time (SRT)**, **Multi-level Feedback (Variable) (FBV)** and **Lottery (LTR)** scheduling algorithms. For each algorithm the program should **list the order and time** of the processes being loaded in the CPU, compute the **turnaround time** and **waiting time** for every process, as well as the average turnaround time and average waiting time. The average values will be consolidated in a table for easy comparison (*check the sample outputs*).

Two sample input data sets and the corresponding outputs have been supplied. Beyond the two sample datasets provided, additional datasets will be used to test your program. The format of the input data will be the same as in the supplied sample files.

2.1 Input Data

Each input data set contains the following information (*check the sample input files for exact format*):

1. Time for running the dispatcher (**DISP:**) which can be zero or a positive integer
2. For each process:
 - i. process id (**PID:**) – as a *string* in the form **pn** where n is a positive integer 1, 2, 3, ... (in order).
 - ii. arrival time (**ArrTime:**) – as a non-negative *integer* representing *unit of time*
 - iii. service time (**SrvTime:**) – as a positive *integer* representing *unit of time*
 - iv. number of tickets (**Tickets**) – as an positive *integer* representing the number of tickets the process holds.
3. A sequence of random numbers to be used (in the given order) only in the lottery (LTR) scheduling algorithm. There should be enough random numbers for the LTR algorithm.

It can be assumed that process P_n will always arrive before or at the same time of process P_{n+1} ; times can be considered to just be whole number '*time units*'.

2.2 Dispatcher

It is assumed that the dispatcher is executed to select the next process to run. The dispatcher should behave as follows:

- i. The time to run the dispatcher (context switching time) is fixed and taken as input (**DISP:**) from the input file. No other time is wasted in switching between processes other than this.

- ii. If the dispatcher starts at $t1$ and finishes at $t2$ (i.e. *time to run the dispatcher is $t2-t1$*) then in that run it will choose from the processes that have arrived at or before $t1$. It will not consider any process that arrives after $t1$ for dispatching in that run.
- iii. If a process **p1** is interrupted at $t1$ and another process **p2** arrives at the same time $t1$ then the newly arrived process **p2** is added in the ready queue first and the interrupted process **p1** is added after that.
- iv. If two processes **px** and **py** have all other properties same (e.g. *arrival time, etc.*) then the tie between them is broken using their ID; i.e. **px** will be chosen before **py** if $x < y$.

3.1. Scheduling Algorithm Details

Some details about the scheduling algorithms are as follows:

FCFS: Standard FCFS scheduling algorithm. Process tickets is ignored in scheduling.

SRT: Standard SRT scheduling algorithm. Process tickets is ignored in scheduling.

FBV: Standard multi-level (3 levels) feedback algorithm with variable time quanta for different queues. The time quanta in different queues (from highest priority to lowest priority) to be 2, 4, and 4 ms, respectively. The bottom queue (i.e. *the lowest priority queue*) is to be treated in round robin fashion. If a process stays (waiting or running) in the lowest priority queue for more than 16 ms then it should be transferred to the highest priority queue again and the whole process repeats for that process.

LTR: Lottery scheduling is a proportional-share scheduling algorithm. Each process has some *tickets* and the percentage of tickets a process has represents the share of the processor time the process should receive. Now the scheduler holds a lottery to determine which process should get to run next. The process has higher percentage of tickets has a higher chance to win the lottery and should run more often. When a process wins the lottery, it is dispatched for a fixed time quantum of 3 ms.

A piece of code snippet (in C++) is given below to show how to run a lottery to choose the winner. You must use the random numbers (in order) in the input file instead of the **.getrandom()** function below to select the winner. If a process could not finish its execution within the allotted time quantum, then it will re-enter the queue at the end.

```

1  // counter: used to track if we've found the winner yet
2  int counter = 0;
3
4  // winner: use some call to a random number generator to
5  // get a value, between 0 and the total # of tickets
6  int winner = getrandom(0, totaltickets);
7
8  // current: use this to walk through the list of jobs
9  node_t *current = head;
10 while (current) {
11     counter = counter + current->tickets;
12     if (counter > winner){
13         break; // found the winner
14     }
15     current = current->next;
16 }
17 // 'current' is the winner: schedule it...
```

3. COMP6240 Additional Task

In addition to the above simulations, you need to write a report on the CPU scheduling algorithm in modern operating systems.

Objective: The objective of this assignment is to conduct a detailed research and analysis of the CPU scheduling algorithms used in various top operating systems, including Windows, Linux, macOS, Android, and iOS. The emphasis will be on understanding the design choices, strengths, weaknesses, and real-world performance of these algorithms. For each OS, students should investigate the specific scheduling algorithm(s) used, its key features, handling of various types of tasks (e.g. real-time VS non-real-time), load balancing and scalability in multi-core systems, mechanism for fairness and efficiency etc. Students will develop a comprehensive report demonstrating their findings.

Description: In this assignment, you will investigate and analyse the CPU scheduling algorithms implemented in three major operating systems, specifically Windows, Linux and macOS. Your report should provide a comparative analysis, discussing the unique approaches each operating system takes to CPU scheduling, and highlight the implications of these choices on system performance, responsiveness, and user experience.

Report Structure and Length: Your report should contain the following sections (i) Introduction (ii) CPU Scheduling in major OS for PCs (iii) Comparative Analysis (iv) Conclusion and (v) References. The length of the report, covering all the sections mentioned before should be 6~10 pages (A4). The report should be written in a formal academic style, with proper citations and references.

The report should be prepared based on your own research and any kind of plagiarism including use of Generative AI in preparing this report should be considered as a serious breach of academic integrity.

4. Submission Requirements

Your submission must also conform to the follow requirements.

4.1. Programming Language

The programming language is Java, versioned as per the University Lab Environment (*Note this is currently a **subversion of Java 17***). You may only use standard Java libraries as part of your submission.

4.2. User Interface

The output should be printed to the console, and strictly following the output samples given in the assignment package. While there are no marks allocated specifically for the Output Format, **there will be a deduction when the result values and result format vary** from those provided.

4.3. Input and Output

Your program will accept data from an input file of name specified as a command line argument. The sample files `datafile1.txt` and `datafile2.txt` (*containing the set1 and set2 data*) are provided to demonstrate the required input file format. **Hint:** the **Java Scanner Library** is something you will likely want to use here!

Your submission will be tested with the above data and will also be tested with other input files.

Your program should output to standard output (*this means output to the Console*). Output should be strictly in the order **FCFS, SRT, FBV, LTR, Summary** (*see the output files*).

The sample files `datafile1_output.txt` and `datafile2_output.txt` (*containing output for datafile1.txt and datafile2.txt respectively*) are provided to demonstrate the required output (*and input*) format which **must be strictly maintained**. **If output is not generated in the required format then your program will be considered incorrect.**

Two Gantt charts are provided to explain the corresponding behaviour of different algorithms and the dispatcher for the two sample data files.

4.4. Use of Generative AI

For completing the programming component of this assignment, you are encouraged to use Generative AI (GenAI) tools. **You must not use GenAI in preparing the reports.** Please check the document “Use of Generative AI in Assignment 1” for information, explanation and possible use cases on how such tools to be used in preparing the solution before you start working on this assignment.

4.5. Mark Distribution

Mark distribution can be found in the assignment draft marking breakdown documents: `24s2_Assign1_Feedback_2240.pdf` & `24s2_Assign1_Feedback_6240.pdf`.

4.6. Deliverable

The following must be observed in your deliverable:

1. Your submission will contain your program source code with documentation and the report (*below*) in the root of the submission. These files will be zipped and submitted in an archive named **c9876543.zip** (where **c9876543** is your student number) – do not submit a `.rar`, or a `.7z`, or etc.
2. Your main class should be **A1.java** your program will compile with the command line **javac A1.java** and your program will be executed by running **java A1 input.txt**.

...where **input.txt** can be *any* filename – **do not hardcode filenames or extensions!**

Note: If your program can not be compiled and executed in the expected manner (*above*) then please add a `readme.txt` (containing any special instructions required to compile and run the source code) in the root of the submission. Please note that such non-standard submissions will be **marked with heavy penalty**.

3. You should write a 2 page (A4) reflective report (name as **Report.pdf**) on the use of Generative AI (GenAI) in competing this assignment. In your reflective note, you should provide a detailed account of your experience and should include, but not limited to, the following elements – (i) overview of the tool(s) used (ii) how the tool(s) was/were integrated in your workflow (iii) what was the quality/correctness of the response generated by the tool(s) and/or how the GenAI tool(s) improved the quality of your solution (iv) how the tool(s) improved your learning experience and problem solving skills (v) what challenges you faced in using those tools and how you solved those challenges (vi) how useful was the tool for your productivity e.g. how much time was saved by using GenAI tool(s) (vii) any ethical consideration or concerns in using GenAI tools in this assessment etc. **GenAI must not be used in preparing this report.**
4. **[COMP6240 students only]** You should submit the research report (name as **Research_Report.pdf**) on the task in Section 3. **GenAI must not be used in preparing this report.**

4.7. Submission Notes

1. Assignments submitted after the deadline (**11:59 pm 06th September 2024**) will have the maximum marks available reduced by 10% per 24 hours.
2. If your assignment is not prepared and submitted following above instructions then you will lose most of your marks despite your algorithms being correct.
3. If you have been granted an extension, please include a copy of the extension approval in as a **PDF document**, called **Extension.pdf**, and place this in the root of your submission, along with your other documents and code.